

Information Security Policy

Application: SpendWise — Personal Spending Analysis Dashboard

Operator: Ryan Snyder (sole developer and operator)

Last reviewed: 2026-05-16

1. Scope and context

SpendWise is a non-commercial personal spending dashboard. It is operated by a single individual and is intended for use only by the operator's friends and family. There are no employees, contractors, or third-party staff with access to user data. No fees are charged to end users. This policy reflects security controls that are appropriate to that scale and have been implemented in the application.

User-supplied data consists of:

- Account credentials (email + bcrypt-hashed password)
- CSV files containing the user's own bank transaction history (uploaded by the user)
- Transaction records pulled via Plaid (transactions, account names, institution names)
- Plaid access_token values, encrypted at rest, used only to read transactions on the user's behalf

The application does not store: bank login credentials, Social Security numbers, full account numbers, payment-card data, or any data that enables money movement.

2. Risk identification

Risks to the data above were identified through threat-modeling during application design. Categories considered:

- **Unauthorized account access** (credential theft, session hijack, broken authorization)
- **Data exposure at rest** (database compromise, host compromise, source-code leak)
- **Data exposure in transit** (network sniffing)
- **Secret leakage** (env vars in version control, log files, error pages)
- **Supply-chain risk** (compromised npm dependencies)
- **Third-party access** (Plaid as a service provider — see §8)

Risks deemed out of scope for this deployment: insider threat from staff (no staff), physical site security beyond standard hosting (deployed to a managed PaaS), DDoS protection beyond hosting-provider defaults.

3. Access control

- **Authentication** is performed by NextAuth (industry-standard library), credentials provider, with passwords hashed using bcrypt (cost factor 10+). Plaintext passwords are never stored or logged.
- **Sessions** are JWT-based, signed with a per-deployment secret (NEXTAUTH_SECRET) held in environment variables.

- **Authorization** is enforced at every API route and server-rendered page: every database query is scoped by the authenticated user's `userId`. There is no admin role, no impersonation feature, and no cross-user data access path.
- **Operator access** to the production database is limited to the operator and is authenticated via a database password held outside source control. Direct database access is used only for backups and incident response.

4. Data protection

At rest

- The application database (PostgreSQL) stores user data with all user-scoped tables keyed by `userId`. Cascading deletes ensure full data removal when a user account is deleted.
- **Plaid access tokens are encrypted with AES-256-GCM** before being written to the database. The encryption key (`PLAID_TOKEN_ENCRYPTION_KEY`) is held in environment variables, separate from the database. A database dump alone does not expose usable Plaid tokens.
- Passwords are stored only as bcrypt hashes; the original passwords cannot be recovered.

In transit

- The application is served over HTTPS in production. HTTP requests are redirected to HTTPS by the hosting provider.
- All calls to Plaid are made server-to-server over HTTPS using the official Plaid Node SDK.

Data minimization

- Uploaded CSV files are stored locally on the application host and are not exposed via any public path. They are deleted from disk when the user deletes the file record.
- Logs do not include passwords, Plaid access tokens, full transaction descriptions, or full CSV contents. This is enforced as a coding rule (`CLAUDE.md §Security Rules`; `SECURITY_REQUIREMENTS.md §Logging`).

5. Key and secret management

All secrets are held in environment variables, not in source control. Specifically:

- `DATABASE_URL` — database credentials
 - `NEXTAUTH_SECRET` — session-token signing key
 - `PLAID_CLIENT_ID`, `PLAID_SECRET` — Plaid API credentials
 - `PLAID_TOKEN_ENCRYPTION_KEY` — symmetric key used to encrypt Plaid access tokens at rest
- `.env` is gitignored. The deployment platform's secret store is used for production values.

Secrets are rotated if known or suspected to be exposed (e.g., accidentally committed to a repository, pasted into an unrelated communication channel, or following a vendor-reported compromise).

6. Software development practices

- Source code is version-controlled (git) with a documented change history.
- Dependencies are managed via `package-lock.json` to ensure reproducible builds.
- TypeScript with strict type-checking is used to reduce a class of bugs that lead to vulnerabilities.
- Code is reviewed before being deployed to production; for a single-developer project, this takes the form of self-review against the project's documented security rules (`SECURITY_REQUIREMENTS.md`, `CLAUDE.md`).
- Coding rules forbid common web vulnerabilities (XSS, SQL injection — Prisma's parameterized queries are used exclusively, no raw SQL with user input) and require validation of any user-supplied data at trust boundaries (uploads, API request bodies).

7. Monitoring and incident response

- Application logs (errors, request failures, Plaid API errors) are reviewed by the operator when problems are reported or when reviewing routine health of the system.
- If an incident is detected — for example, suspected unauthorized account access, a leaked secret, or a Plaid-side notification of a compromised item — the response procedure is:
 - 1 Rotate the affected secret(s) immediately (database password, NextAuth secret, Plaid encryption key, Plaid API secret).
 - 2 For Plaid-specific incidents, call `itemRemove` on affected items to invalidate access tokens with Plaid.
 - 3 Notify affected end users (friends/family members) directly.
 - 4 If user data appears to have been accessed by an unauthorized party, notify Plaid via the appropriate support channel.
- Because the user base is small and known to the operator, incident notification is direct and personal rather than mass-broadcast.

8. Third-party services and vendor risk

The application uses the following third-party services that have access to user-related data:

Vendor	Purpose	Data shared
Plaid	Bank connection and transaction retrieval	Account/transaction data, per user consent obtained via Plaid Link
Hosting provider	Application and database hosting	All application data

Each vendor was selected on the basis of public security documentation (SOC 2 reports, published security pages) at the time of integration. Plaid's role is governed by Plaid's end-user privacy policy, which is shown to and accepted by every end user during the Plaid Link flow.

9. End-user consent and rights

- Each user creates their own account with their own credentials.
- Each user explicitly initiates each Plaid Link flow themselves and consents to Plaid's privacy policy at that time.
- A user can disconnect any Plaid connection from the application UI at any time, which calls Plaid's `itemRemove` to invalidate the corresponding access token.
- A user can delete uploaded CSV files at any time, which removes the file and all parsed transactions.
- Deleting a user account cascades to delete all of that user's files, transactions, Plaid items, and Plaid accounts.

10. Policy review

This document is reviewed when material changes are made to the application (new integrations, schema changes affecting sensitive data, vendor changes) and at least annually.